

PUBLIC

This template has been updated according to the latest data classification guidelines

Learn more

O4AA -> AWS Bedrock Agents -> Okta Agent Gateway -> GitHub MCP Demo

David Edwards
Product Acceleration Specialist, Okta

© Okta, Inc. and/or its affiliates. All rights reserved.

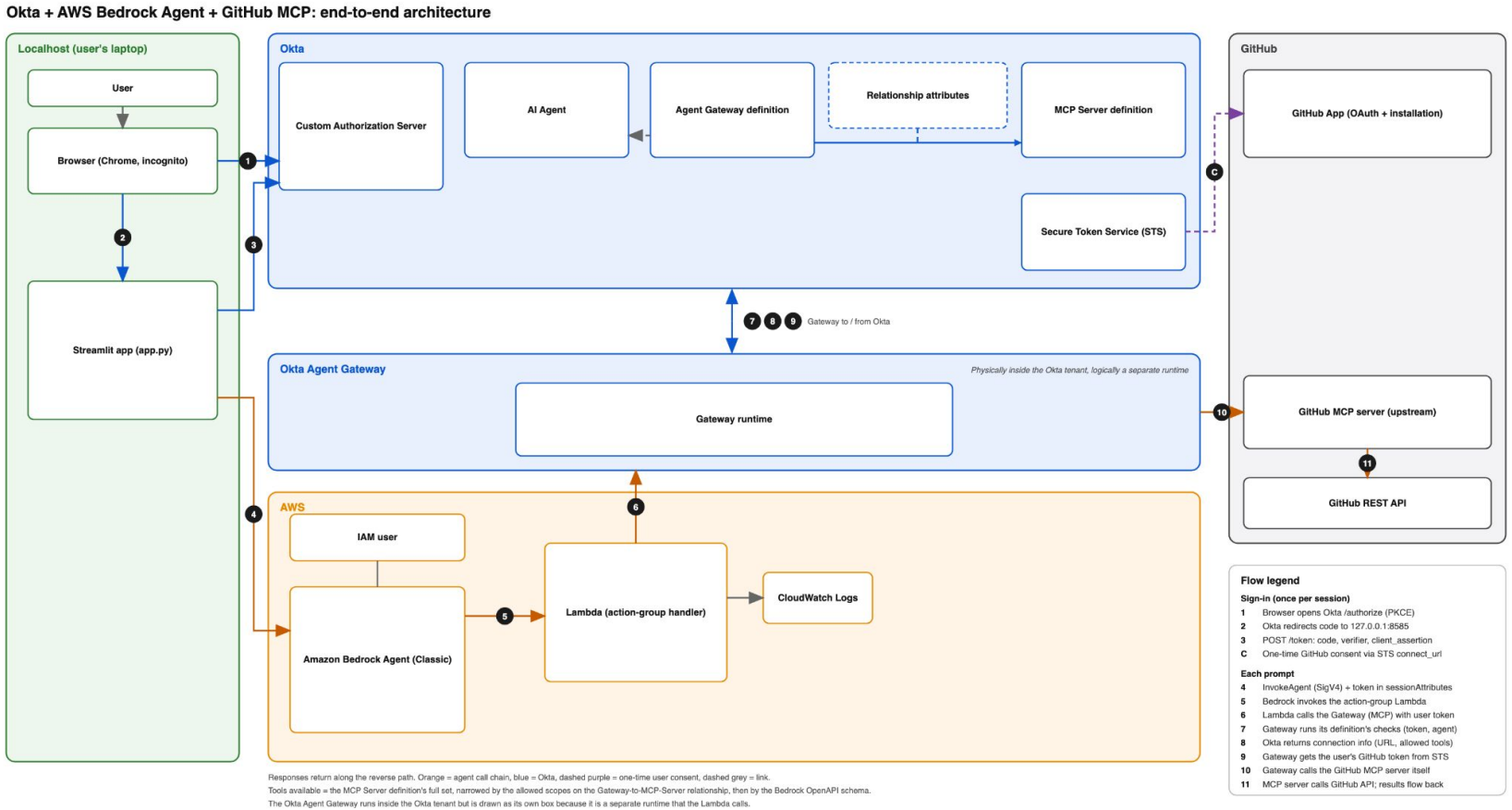
DATA CLASSIFICATION: OKTA, INC. PUBLIC

Agenda

1. High-level Architecture of solution
2. Considerations for O4AA/Agent Gateway with Bedrock
3. Demo architecture
4. Okta Components
    a. With Flows and Authentication and Authorization
    b. With Authentication and Authorization summary
5. Changes Required to Add O4AA

okta © Okta, Inc. and/or its affiliates. All rights reserved.

High-Level Demo Architecture



. PUBLIC

Considerations for O4AA/Agent Gateway with Bedrock

Bedrock Agents (Classic) cannot call MCP services

- Their only tool mechanism is an action group (Lambda or return-of-control)
- Need to call Lambda function to make tool calls to an external MCP Server
- Use of the AWS Bedrock AgentCore Gateway adds a hop with no benefit when the Okta Agent Gateway is already the MCP front door
- To call Bedrock AgentCore Gateway, need to use AgentCore Agents (then trigger a Lambda function to do any needed token exchange if Okta Agent Gateway is not used)

Bedrock Agents (Classic) and Bedrock AgentCore Harness environments are serverless

- They cannot invoke a browser, i.e. Classic cannot host a redirect URI or hold interactive state and AgentCore cannot interact with the user directly. It only acts as the “backend” component of an agent performing the orchestration and interaction with LLM and services.
- Need an external app as the “frontend” app.
    - The localhost app owns sign-in and the callback, and the Lambda is stateless (it re-initializes the MCP session on every call).
    - Default mode with just agents in Bedrock / AgentCore Harness would be a machine2machine model

okta © Okta, Inc. and/or its affiliates. All rights reserved.

Considerations for O4AA/Agent Gateway with Bedrock

Differences between Bedrock Agents (Classic) and Bedrock AgentCore WRT Imports

- Both can be imported, and Okta's current docs list both with resource connection support.
- The import is a read-only inventory (bedrock:GetAgent, bedrock:ListAgents). It doesn't discover connections from AWS.
- With Classic, Okta's governed connection attaches to the agent identity that signs in (the linked app), not to the Bedrock agent resource.
- The Bedrock agent is effectively a runtime for the Lambda.

Tool definitions are static

- The agent only knows the tools in the action group's OpenAPI schema. It does not discover MCP tools.

Bedrock does not have a mechanism to pass Okta identity across to services (like AWS Bedrock AgentCore Gateway)

- Classic InvokeAgent authenticates with IAM only, so the user's identity has to travel as data.
- AgentCore Runtime differs. It can accept an Okta JWT as inbound auth, but it's outbound tool calls then run as the agent's own principal, which breaks attribution.

okta © Okta, Inc. and/or its affiliates. All rights reserved.

DATA CLASSIFICATION: OKTA, INC. PUBLIC

Considerations for O4AA/Agent Gateway with Bedrock

There are two authorization layers:

- IAM decides who can invoke the agent, at app level. Okta decides what's done for which user, at the Gateway. Neither enforces the other.
- There are also two audit trails: CloudTrail shows the IAM user, and the Okta System Log shows the user and the agent.

Other considerations

- **Token handling.** The bearer token rides through the Bedrock session and the Lambda. Keep it out of logs and traces. It's short-lived and there's no refresh path in the agent, so the frontend must handle expiry.
- **Consent has to be surfaced.** When the Gateway returns connect_url, something must relay it to the user. Design the Lambda and frontend for that response.
- **Error handling.** The Gateway returns errors as HTTP 200. A non-2xx from the Lambda becomes dependencyFailedException and hides the cause, so the Lambda has to translate errors into normal responses.
- **Session IDs.** Use one per user and conversation, never shared between users.

okta © Okta, Inc. and/or its affiliates. All rights reserved.

DATA CLASSIFICATION: OKTA, INC. PUBLIC

Considerations for O4AA/Agent Gateway with Bedrock

Minimum AWS Agent code change model involves:

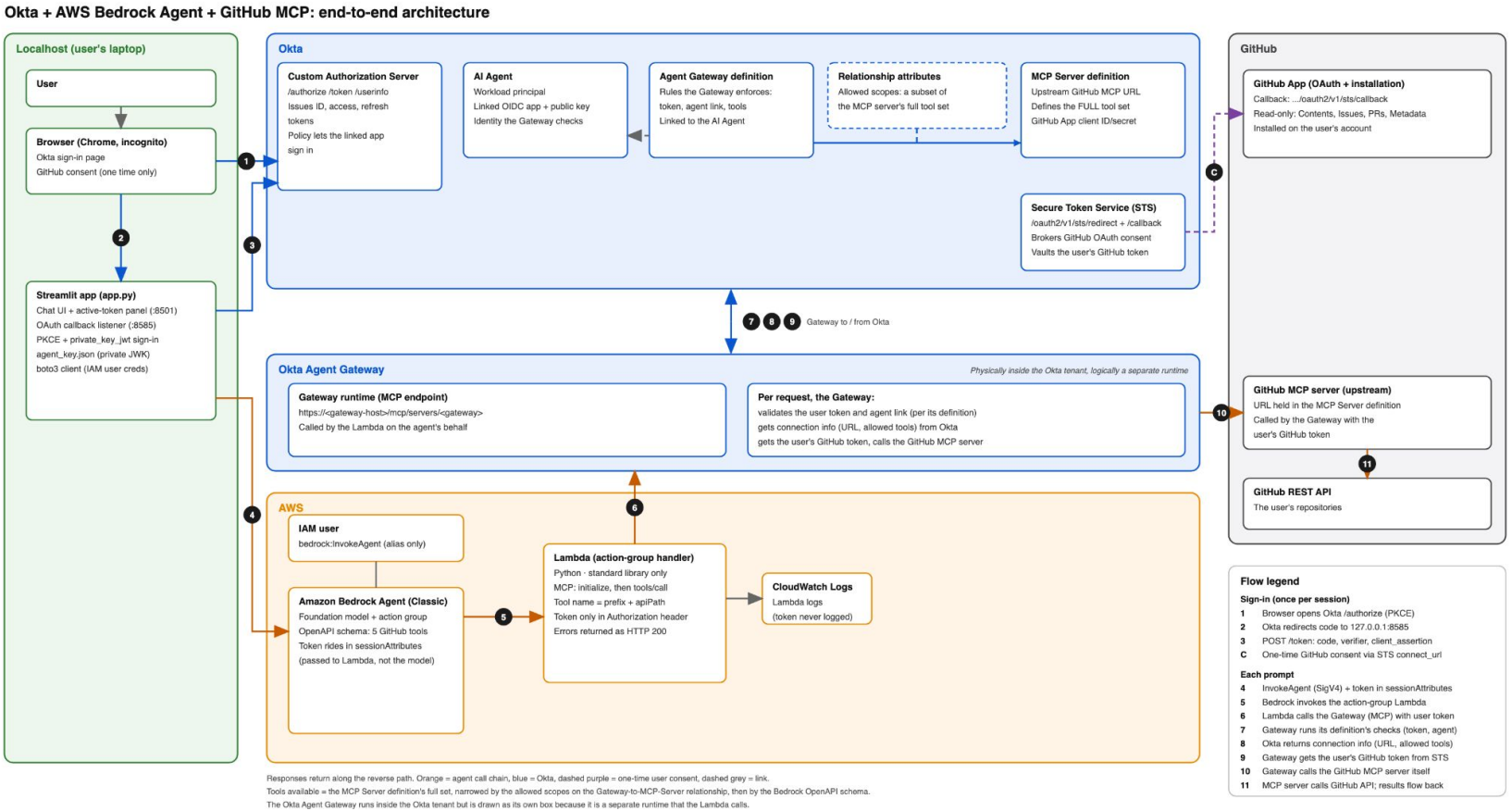
- An external “frontend” app to perform the user authentication (OAuth+PKCE).
- App calls the Bedrock Agent using AWS IAM creds, but passes Okta access token in sessionAttributes
  - Bedrock agent has it’s (action group) schema to define all the downstream MCP Server tools and arguments required for making the tool calls, but has nothing defined for the Okta Agent Gateway or other Okta objects
  - Bedrock agent is not coded to make token exchange calls with Okta (or any other IdP)
- A bespoke Lambda function is required to make the tool calls to the Okta Agent Gateway
  - It consumes the sessionAttributes (access token) and uses the values to make the gateway call
  - It knows how to construct the call to Okta Agent Gateway and process the response
  - It doesn’t need to know about specific tools unless you want to make it a router for multiple gateways

Security benefit for this model.

- No Okta or GitHub secrets live in AWS.
- The Lambda forwards the user's token and the Gateway does the exchange.

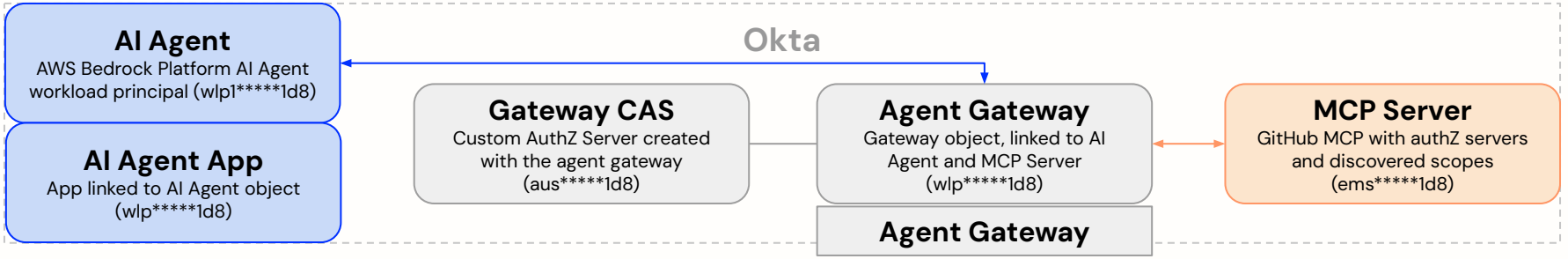
© Okta, Inc. and/or its affiliates. All rights reserved.

Demo Architecture



. PUBLIC

Okta Components



AI Agent object for the AWS Bedrock Platform (not the specific agent).

Linked app (existing or auto-created) defining who can use the app/agent and the authentication policies that apply to the users.

Custom Authorization Server (CAS) created automatically when the Agent Gateway object is created.

Access policy to allow Authorization Code grant for a group of users.

Allowing the AI Agent users to access the Agent Gateway

Agent Gateway object which creates the **Agent Gateway** instance.

Linked to the AI Agent object (which agents can call the Gateway) and which resources (like MCP Servers) the gateway can call.

Tool subset defined on the Agent Gateway for each resource.

MCP Server object to define downstream resources.

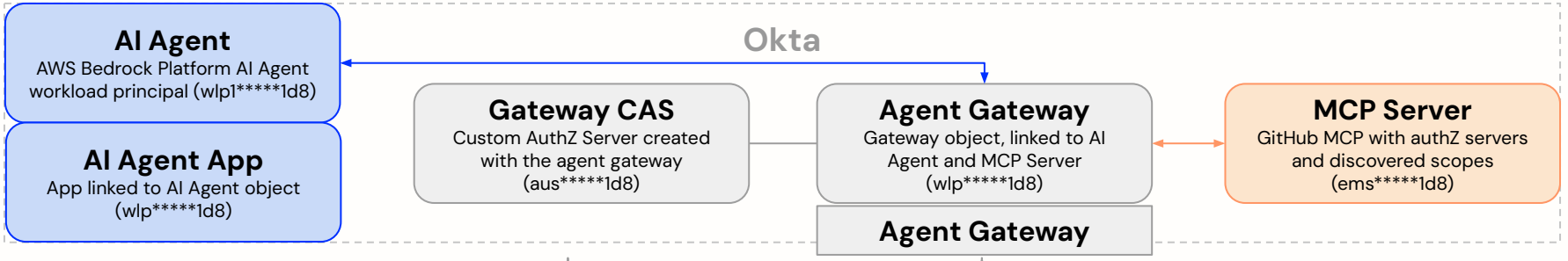
Can be external auth (their own AS) or Okta managed (Okta CAS).

Defines connection to connect to the resource and discover the tools the resource can access.

okta © Okta, Inc. and/or its affiliates. All rights reserved.

DATA CLASSIFICATION: OKTA, INC. PUBLIC

Okta Components - Flow with AuthN & AuthZ



Demo front-end app (app.py) launches browser to GET against the CAS /authorize endpoint with client_id of AI Agent app.

If needed Okta authenticates the user with apps authentication policy (incl. MFA).

Authorization code returned to app.

App then makes a POST to the CAS /token endpoint to generate ID, access and refresh tokens. Their cid is the agents client ID which the gateway later relies on.

Client passes prompt, access token and client ID*, session ID, agentID and agentAliasID in Bedrock call.

*sessionAttributes used to pass access token and client ID

Called **agent** (classic) is just a shell, connected to a LLM and an action group. Action group has tools schema and Lambda group to call. Agent knows nothing of Okta.

The **Lambda function** calls the Okta Agent Gateway for the tool call and sends the response back to the agent. It doesn't know about the tools, just passes the calls through. It uses the access token from the session attributes.

Users access token is passed to the **Agent Gateway** in the header, no GitHub creds. Tool call & args passed in the body.

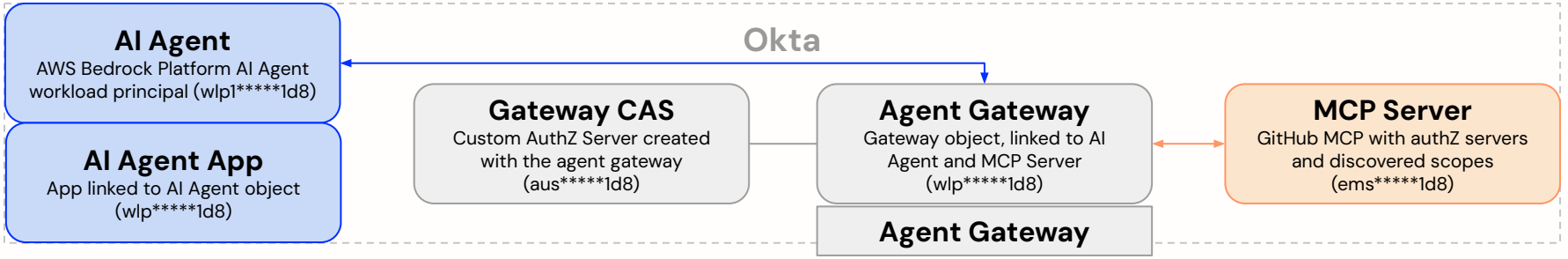
It checks token validity, tool allowed and connection lookup. GitHub is an OAuth STS connection so it will process creds from the token store.

It calls the **MCP Server** and passes results back to the Lambda function.

okta © Okta, Inc. and/or its affiliates. All rights reserved.

DATA CLASSIFICATION: OKTA, INC. PUBLIC

Okta Components - AuthN & AuthZ Summary



Agent App assignment determines who (Okta users) can access the Agent (in this case Bedrock itself).

Agent App authentication policy determines authN requirements (like MFA)

CAS mints an access token that contains the user (sub), CAS (iss), Agent Gateway (aud), Agent (cid & act).

It's saying this user, in the context of this agent would like to access the Agent Gateway.

AI Agent <-> Gateway <-> MCP Server relationship determines what agent can call the Gateway and what the Gateway can call.

The gateway will use the access token to check.

Gateway <-> MCP Server relationship has subset of tools allowed.

MCP Server connection provides a list of tools that could be called.

Stored access/refresh tokens in token store for service account.

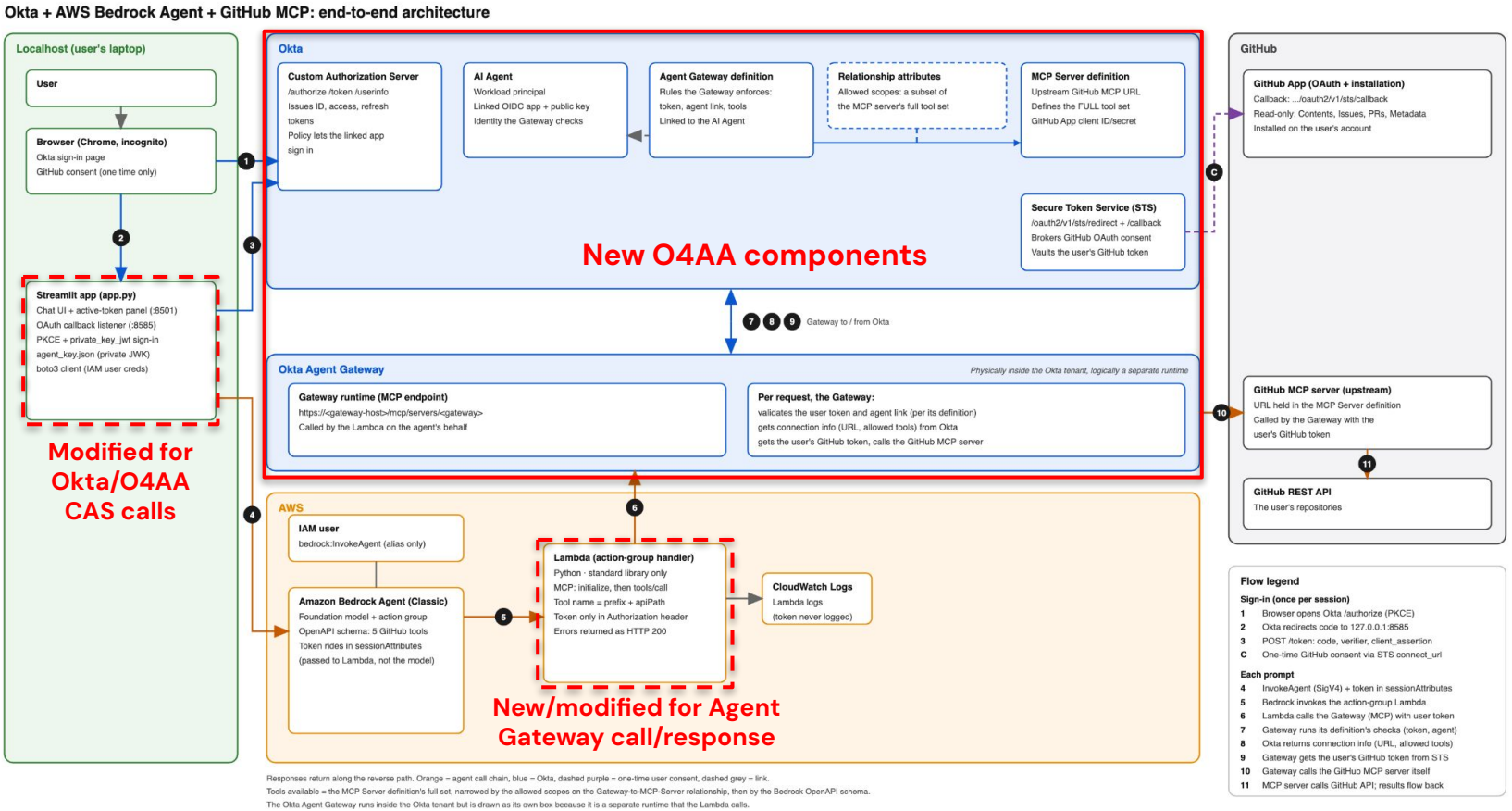
If tool not allowed, Gateway will return an error to the caller.

Call from frontend app to Bedrock is using an AWS IAM account, not Okta credentials.

okta © Okta, Inc. and/or its affiliates. All rights reserved.

DATA CLASSIFICATION: OKTA, INC. PUBLIC

What Changes Required to Add O4AA



. PUBLIC

Thank you!

okta © Okta, Inc. and/or its affiliates. All rights reserved.

DATA CLASSIFICATION: OKTA, INC. PUBLIC